Rohan Sharma

+91 6395833628 rohansharmaksp222@gmail.com LinkedIn Github

EDUCATION

BMS Institute of Technology and Management Bangalore, Karnataka
B.E. in Artificial Intelligence and Machine Learning; CGPA: 8.35 Aug. 2021 – July. 2025

EXPERIENCE

- **Mejuvante.ai** *June 2025 – Present*
  Software Engineer – Backend Bengaluru, India
  - Architected **cost-optimized AWS deployment infrastructure** for multiple applications, designing containerized workloads using **Docker, ECS, and EKS**, with observability via **CloudWatch** and scalable asset delivery through **S3 and CloudFront**.
  - Built **production-grade CI/CD pipelines** using **GitHub Actions, AWS CodeBuild, and Jenkins**, implementing automated build, testing, and deployment workflows with **environment-aware pipelines for dev and production**.
  - Established **Git-based release engineering strategy (feature--dev--main)** with automated validation pipelines, enabling controlled promotion of code across environments and reliable production releases.
  - Engineered **high-concurrency backend services using Python and FastAPI**, incorporating **rate limiting, connection pooling, database replicas, and load balancing** to eliminate single points of failure and improve system resilience.
  - Designed and integrated **LLM-powered backend systems**, implementing **Retrieval-Augmented Generation (RAG) pipelines with vector databases** to enable accurate natural-language querying over domain datasets.
  - Built **internal automation platforms and infrastructure tooling**, including workflow automation and **self-hosted services on a Raspberry Pi cluster** to run internal workloads and reduce cloud operational costs.

- **Marque Magic Pvt Ltd** *Jan 2025 – Apl 2025*
  Software Engineer- cloud (Intern) Bengaluru, India
  - **Cloud Region Build & Control/Data Plane Architecture:**
    - Developed **Python-based automation tooling** for cloud environment validation and provisioning, streamlining deployment readiness checks for containerized services.
    - Performed **API and integration testing for ECS-based microservices**, validating service communication and improving release reliability.
    - Contributed to **GitLab CI/CD pipelines**, enabling automated build, testing, and deployment workflows across staging environments.
    - Diagnosed system defects and collaborated with engineering teams to resolve **deployment and service orchestration issues**, improving platform stability.
    - Gained hands-on exposure to **AWS cloud infrastructure, containerized microservices, and production deployment workflows**.

PROJECTS

- **Distributed Ride Booking System | Link**
  Node.js, Express, Docker, Kubernetes, RabbitMQ, Redis, API Gateway
  - Designed a **microservices-based ride booking platform** modeling real-world ride platforms, with independently deployable **user, ride, payment, and notification services**.
  - Implemented **API Gateway routing, rate limiting, and load balancing**, enabling controlled traffic management and improved system resilience under concurrent requests.
  - Built **event-driven inter-service communication using RabbitMQ** and **Redis-based caching** for real-time ride updates and reduced request latency.
  - Applied backend reliability patterns including **asynchronous request handling, centralized error handling, and service-level fault tolerance**.
  - Containerized all services using **Docker** and orchestrated deployments via **Kubernetes**, enabling scalable distributed service execution.

- **Metadata Collection Service | Link**
  Python, FastAPI, MongoDB, Docker Compose, Pytest
  - Built a **metadata collection service** that retrieves and stores URL headers, cookies, and page source, exposing REST APIs for metadata ingestion and retrieval.
  - Designed a **cache-first retrieval workflow**, where existing metadata is returned instantly while cache misses trigger asynchronous background collection.

- Implemented **non-blocking background workers** to fetch metadata without delaying API responses, ensuring responsive request handling.
- Containerized the service using **Docker Compose** with MongoDB, enabling reproducible environments and simplified local deployment.
- Structured the codebase with **separation between API, service, and persistence layers**, improving maintainability and extensibility.

- **Talk2Data | Link**

FastAPI, PostgreSQL, ChromaDB, Docker, AWS, Traefik, LLM

- **FastAPI, PostgreSQL, ChromaDB, Docker, AWS, Traefik, LLM**
- Built an **LLM-powered data querying platform** enabling natural-language interaction with relational databases, improving developer productivity and data accessibility.
- Implemented a **Retrieval-Augmented Generation (RAG) pipeline** using ChromaDB vector embeddings to map database schemas and generate context-aware SQL queries.
- Designed a **modular FastAPI backend** with query validation, debugging pipelines, and role-based access control to ensure secure query execution.
- Optimized **SQL schema design and query workflows** to improve reliability and maintainability of generated queries.
- Containerized services with **Docker and deployed behind Traefik reverse proxy** with SSL-secured routing.

SKILLS

- **Programming Languages:** Python, JavaScript, C++, SQL
- **Backend & APIs:** FastAPI, Node.js, Express.js, REST APIs, Microservices Architecture, Asynchronous Processing
- **Databases & Caching:** PostgreSQL, Redis, Database Schema Design, Query Optimization
- **Distributed Systems & Messaging:** API Gateway, RabbitMQ, Rate Limiting, Load Balancing
- **Cloud & DevOps:** AWS (EKS, EC2, IAM, Lambda), Docker, Kubernetes, Terraform, Jenkins, Git, GitHub, Postman
- **Core CS Fundamentals:** Data Structures & Algorithms (200+ problems solved), Operating Systems, Computer Networks, Database Systems, Software Engineering

---

Research

Early Plant Disease Prediction using CNN and Sensor Data

Manuscript submitted to **IEEE (Under Review)

- Designed a machine learning system to detect **early plant disease risk** using leaf images and environmental sensor data.
- Implemented CNN-based image classification combined with temperature, humidity, and soil moisture inputs.
- Built predictive models to identify disease likelihood before visible symptoms appear.